

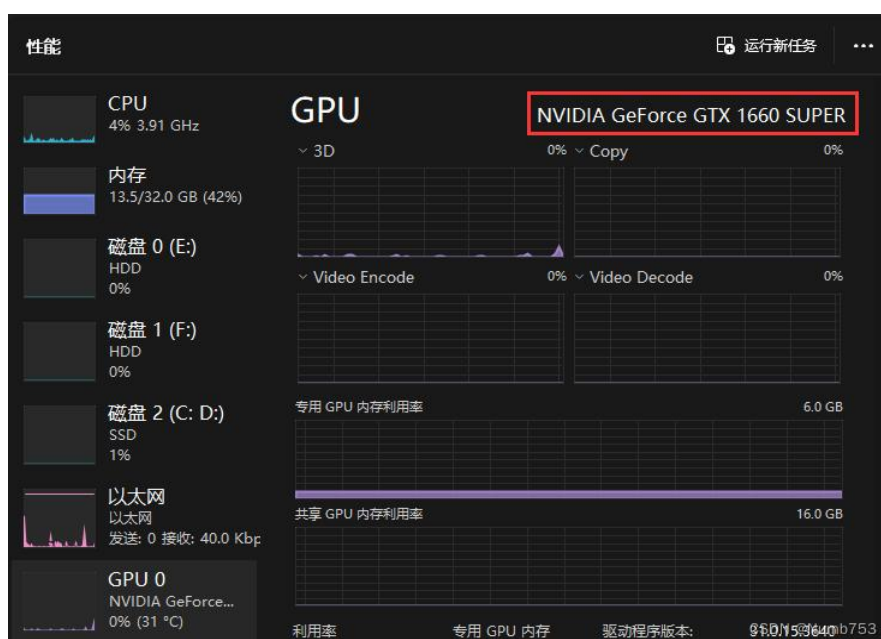
目标检测：YOLOv11(Ultralytics)环境配置

-csdn:

https://blog.csdn.net/qq_67105081/article/details/143270109?spm=1001.2014.3001.5502

1. 查看电脑状况

深度学习的训练对于电脑显卡要求较高，若电脑没有独立显卡（NVIDIA 卡）或者是 AMD 的显卡无法使用 GPU 进行训练，需用 CPU 进行训练，会慢很多。若不清楚电脑有无显卡可以打开任务管理器，点击性能往下找 GPU，就可以看到是否有独立显卡，不清楚是否为独立显卡则搜索型号。



若有独立显卡则安装 GPU 版本的 pytorch，若无独立显卡则安装 CPU 版本的 pytorch。

2. 安装所需软件

推荐安装 [Anaconda3](#)

[\[https://so.csdn.net/so/search?q=Anaconda3&spm=1001.2101.3001.7020\]](https://so.csdn.net/so/search?q=Anaconda3&spm=1001.2101.3001.7020)+Pycharm，都需要加入环境变量，会安装或者安装过了则跳过这一步骤。

2.1 Anaconda3 安装

Anaconda3 由于是国外网站下载较慢，推荐通过清华镜像源安装。

Index of /anaconda/archive/ | 清华大学开源软件镜像站 | Tsinghua Open Source MirrorIndex of /anaconda/archive/ | 清华大学开源软件镜像站，致力于为国内和校内用户提供高质量的开源软件镜像、Linux 镜像源服务，帮助用户更方便地获取开源软件。本镜像站由清华大学 TUNA 协会负责运行维护。
<https://mirrors.tuna.tsinghua.edu.cn/anaconda/archive/?C=M&O=D>
<https://mirrors.tuna.tsinghua.edu.cn/anaconda/archive/?C=M&O=D>
<https://mirrors.tuna.tsinghua.edu.cn/anaconda/archive/?C=M&O=D>
<https://mirrors.tuna.tsinghua.edu.cn/anaconda/archive/?C=M&O=D>
<https://mirrors.tuna.tsinghua.edu.cn/anaconda/archive/?C=M&O=D>

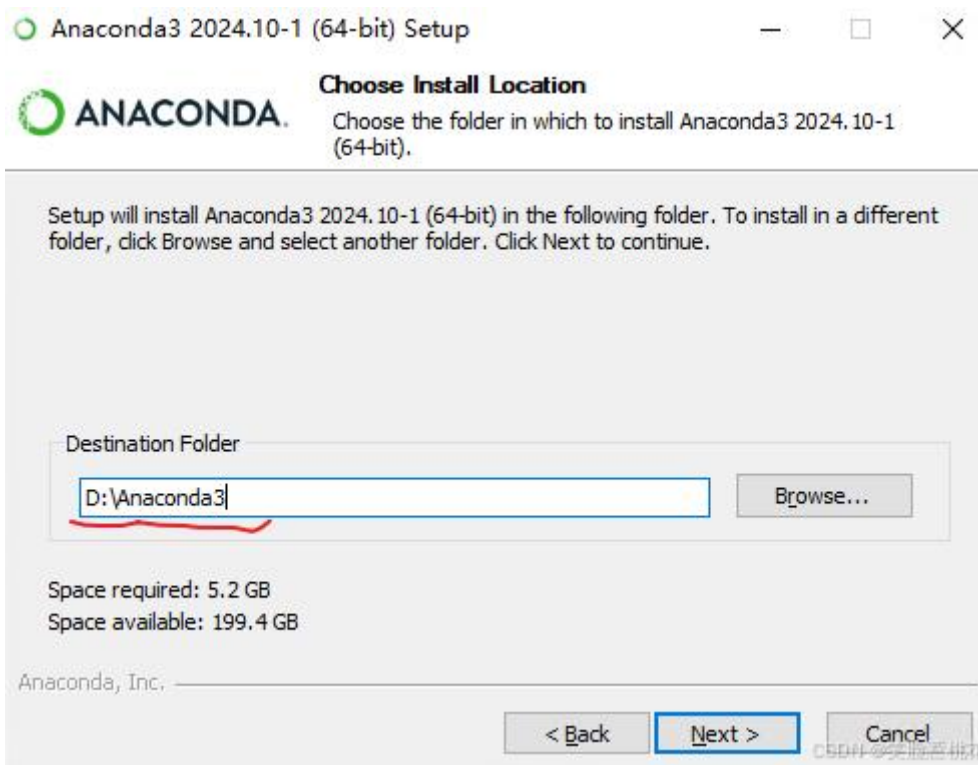
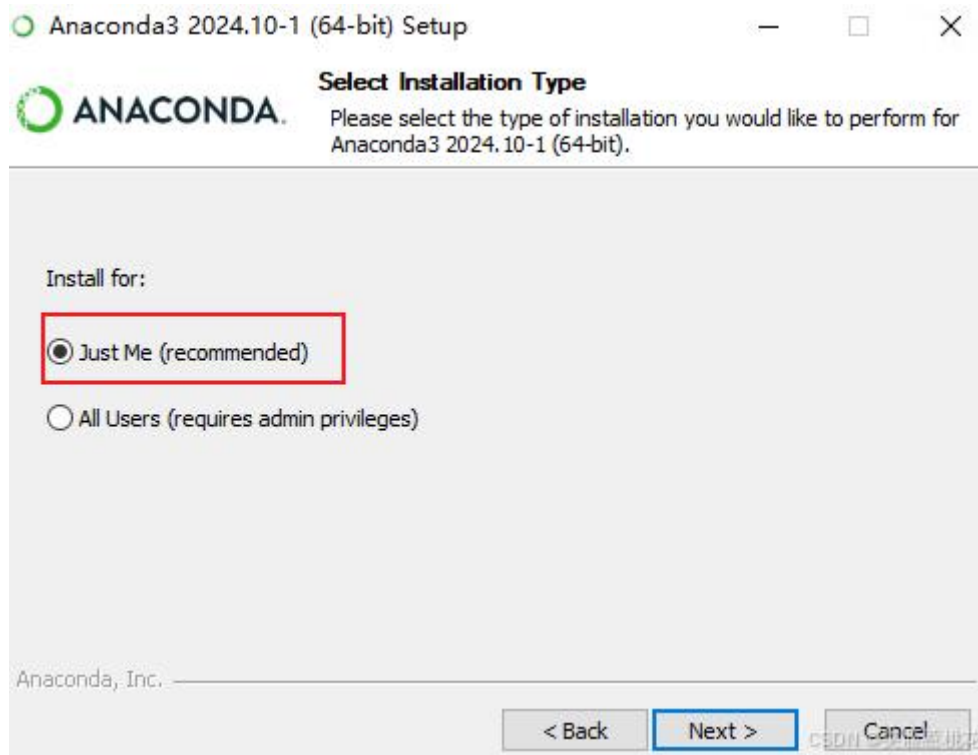
选择带有 Anaconda3...Windows...exe 字样的进行下载。

Index of /anaconda/archive/ Last Update: 2024-10-27 12:56

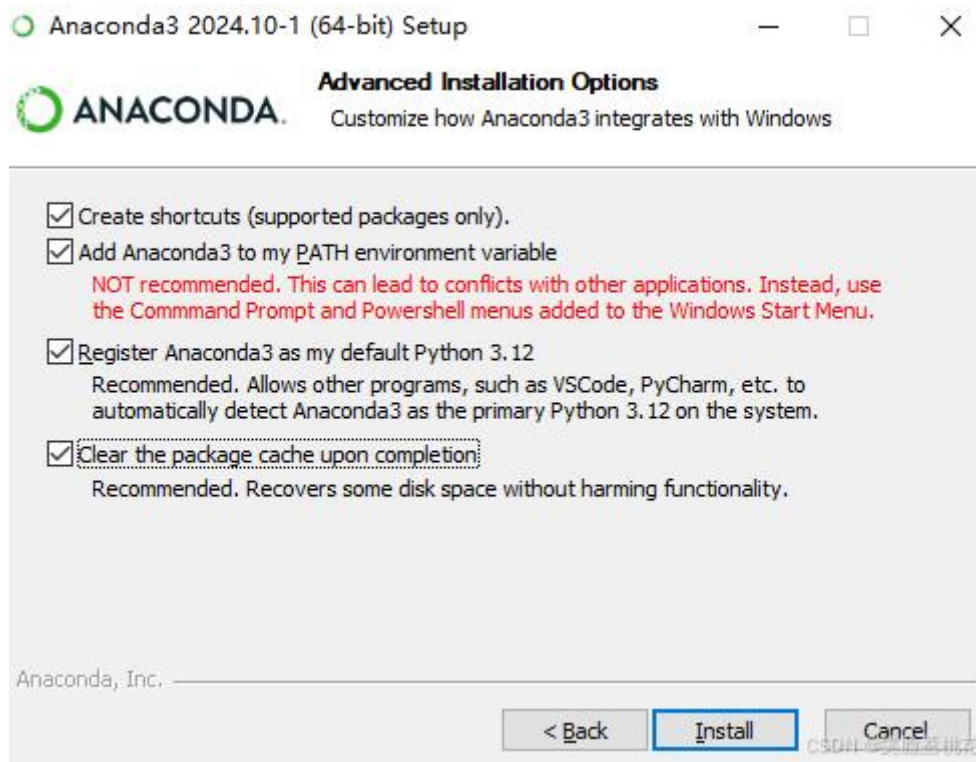
File Name ↓	File Size ↓	Date ↓
Parent directory/	-	-
Anaconda3-2024.10-1-MacOSX-x86_64.sh	778.5 MiB	2024-10-24 01:52
Anaconda3-2024.10-1-MacOSX-x86_64.pkg	776.0 MiB	2024-10-24 01:52
Anaconda3-2024.10-1-Windows-x86_64.exe	950.5 MiB	2024-10-24 01:52
Anaconda3-2024.10-1-Linux-aarch64.sh	843.2 MiB	2024-10-24 01:52
Anaconda3-2024.10-1-MacOSX-arm64.pkg	744.6 MiB	2024-10-24 01:52
Anaconda3-2024.10-1-Linux-s390x.sh	441.2 MiB	2024-10-24 01:52
Anaconda3-2024.10-1-MacOSX-arm64.sh	747.2 MiB	2024-10-24 01:52
Anaconda3-2024.10-1-Linux-x86_64.sh	1.0 GiB	2024-10-24 01:52
Anaconda3-2024.06-1-Windows-x86_64.exe	912.3 MiB	2024-06-27 04:49
Anaconda3-2024.06-1-MacOSX-x86_64.sh	737.2 MiB	2024-06-27 04:49
Anaconda3-2024.06-1-MacOSX-x86_64.pkg	734.7 MiB	2024-06-27 04:49
Anaconda3-2024.06-1-Linux-x86_64.sh	1007.9 MiB	2024-06-27 04:48
Anaconda3-2024.06-1-MacOSX-arm64.sh	707.3 MiB	2024-06-27 04:48
Anaconda3-2024.06-1-Linux-aarch64.sh	800.6 MiB	2024-06-27 04:48
Anaconda3-2024.06-1-Linux-s390x.sh	425.8 MiB	2024-06-27 04:48
Anaconda3-2024.06-1-MacOSX-arm64.pkg	704.7 MiB	2024-06-27 04:48
Anaconda3-2024.02-1-Windows-x86_64.exe	904.4 MiB	2024-02-27 06:01
Anaconda3-2024.02-1-MacOSX-x86_64.sh	731.2 MiB	2024-02-27 06:01
Anaconda3-2024.02-1-MacOSX-x86_64.pkg	728.7 MiB	2024-02-27 06:01

CSDN @笑脸惹桃花

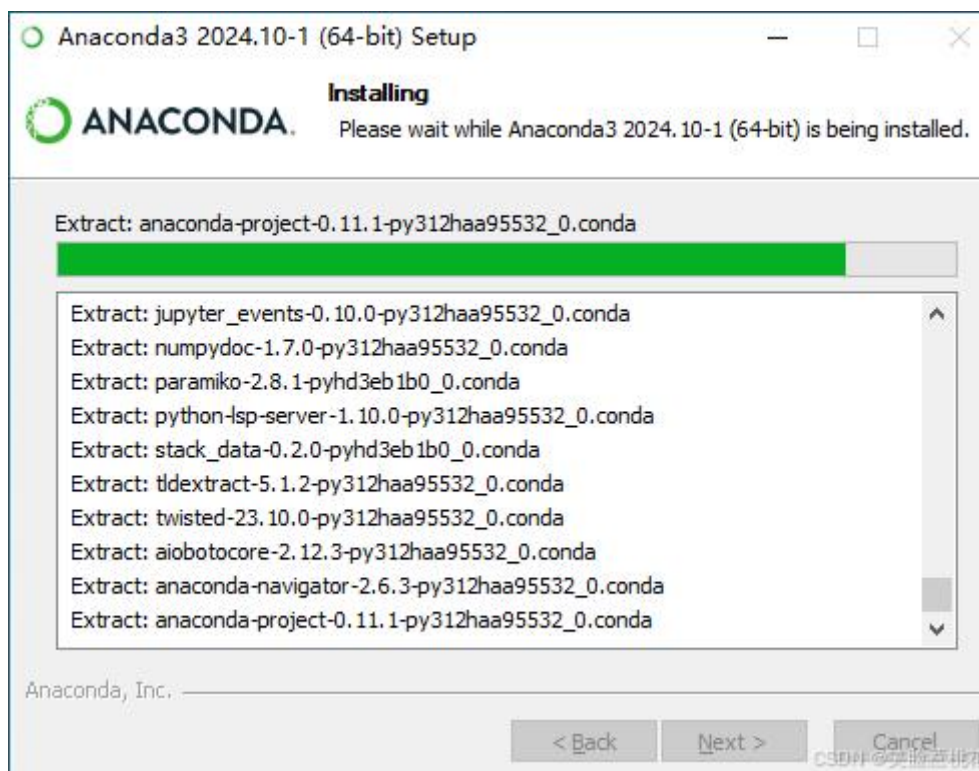
下载完成之后打开 Anaconda3 进行安装，一直点下一步，选 Just Me，安装路径不建议安装到 c 盘，可以直接复制粘贴修改到 D:\Anaconda3，也可以修改到其他路径，最好纯英文路径。



点击下一步后，需要选择添加到环境变量，如下图前三个一定要勾选，也可以按照我图中全选。



点击 install 安装后耐心等待进度条满安装完成即可，进度较慢，耐心等待。



2.2 Pycharm 安装

<https://so.csdn.net/so/search?q=Pycharm%E5%AE%89%E8%A3%85&spm=1001.2101.3001.7020>

安装 Pycharm 可以直接去官网下载，速度较快。

下载 PyCharm: JetBrains 出品的用于数据科学和 Web 开发的 Python IDE 现在在最新版本的 PyCharm，适用于 Windows、macOS 或 Linux。

<https://www.jetbrains.com/zh-cn/pycharm/download/?section=windows>
<https://www.jetbrains.com/zh-cn/pycharm/download/?section=windows>
<https://www.jetbrains.com/zh-cn/pycharm/download/?section=windows>
<https://www.jetbrains.com/zh-cn/pycharm/download/?section=windows> [https://www.jetbrains.com/zh-cn/pycharm/download/?section=windows]

往下拉下载第二个 Community Edition 社区免费版就可以，也可以[网盘下载](https://pan.quark.cn/s/fa7aaa8a8a95) [https://pan.quark.cn/s/fa7aaa8a8a95] 相同版本。



下载完之后打开安装，点下一步，遇到选择路径 修改路径到 D 盘或者其它除 C 盘外的文件夹，可以建个自己喜欢的英文名字。



需要勾选这些选项，五角星必勾选，建议全选。



再点下一步，直接安装就可以了，耐心等待进度条满安装完成即可。



3. 安装环境

3.1 安装 cuda 及 cudnn

3.1.1 下载及安装 cuda



安装完成后可以再次在 cmd 里输入命令：nvcc -V 查看，如下显示即安装成功

```
C:\Users\...\P>nvcc -V
nvcc: NVIDIA (R) Cuda compiler driver
Copyright (c) 2005-2022 NVIDIA Corporation
Built on Wed_Sep_21_10:41:10_Pacific_Daylight_Time_2022
Cuda compilation tools, release 11.8, V11.8.89
Build cuda_11.8.r11.8/compiler.31833905_0
```

CSDN @笑臉惹桃花

3.1.2 cudnn 安装

进入 cudnn 官网，选择合适版本的文件。

<https://developer.nvidia.com/rdp/cudnn-archive>
<https://developer.nvidia.com/rdp/cudnn-archive>
<https://developer.nvidia.com/rdp/cudnn-archive>
<https://developer.nvidia.com/rdp/cudnn-archive>
[\[https://developer.nvidia.com/rdp/cudnn-archive\]](https://developer.nvidia.com/rdp/cudnn-archive)

进入后在文件列表中选择 cudnn 版本与上面 cuda 安装相匹配的版本。

cuDNN Archive

NVIDIA cuDNN is a GPU-accelerated library of primitives for deep neural networks.

Download cuDNN v8.9.7 (December 5th, 2023), for CUDA 12.x
Download cuDNN v8.9.7 (December 5th, 2023), for CUDA 11.x
Download cuDNN v8.9.6 (November 1st, 2023), for CUDA 12.x
Download cuDNN v8.9.6 (November 1st, 2023), for CUDA 11.x
Download cuDNN v8.9.5 (October 27th, 2023), for CUDA 12.x
Download cuDNN v8.9.5 (October 27th, 2023), for CUDA 11.x
Download cuDNN v8.9.4 (August 8th, 2023), for CUDA 12.x
Download cuDNN v8.9.4 (August 8th, 2023), for CUDA 11.x
Download cuDNN v8.9.3 (July 11th, 2023), for CUDA 12.x
Download cuDNN v8.9.3 (July 11th, 2023), for CUDA 11.x
Download cuDNN v8.9.2 (June 1st, 2023), for CUDA 12.x
Download cuDNN v8.9.2 (June 1st, 2023), for CUDA 11.x
Download cuDNN v8.9.1 (May 5th, 2023), for CUDA 12.x
Download cuDNN v8.9.1 (May 5th, 2023), for CUDA 11.x
Download cuDNN v8.9.0 (April 11th, 2023), for CUDA 12.x
Download cuDNN v8.9.0 (April 11th, 2023), for CUDA 11.x

CSDN @笑脸惹桃花

下载 Windows 版本的压缩包文件。

[Download cuDNN v8.9.7 \(December 5th, 2023\), for CUDA 11.x](#)

Local Installers for Windows and Linux, Ubuntu(x86_64, armsbsa)

[Local Installer for Windows \(Zip\)](#)

[Local Installer for Linux x86_64 \(Tar\)](#)

[Local Installer for Linux PPC \(Tar\)](#)

[Local Installer for Linux SBSA \(Tar\)](#)

[Local Installer for Debian 11 \(Deb\)](#)

[Local Installer for Ubuntu18.04 x86_64 \(Deb\)](#)

[Local Installer for Ubuntu20.04 x86_64 \(Deb\)](#)

[Local Installer for Ubuntu22.04 x86_64 \(Deb\)](#)

[Local Installer for Ubuntu20.04 aarch64sbsa \(Deb\)](#)

[Local Installer for Ubuntu22.04 aarch64sbsa \(Deb\)](#)

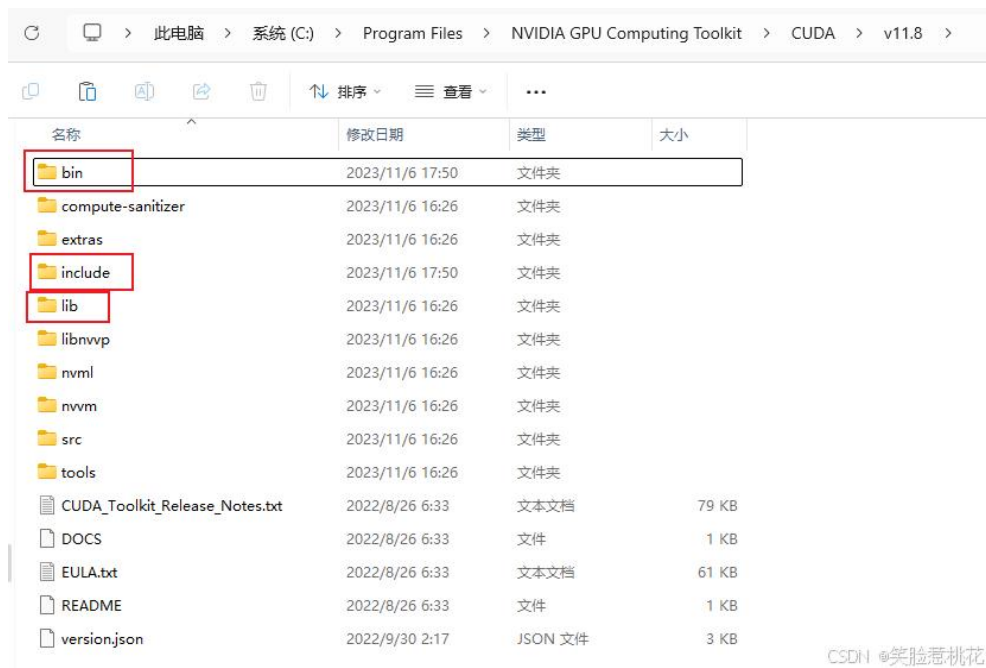
[Local Installer for Ubuntu20.04 cross-sbsa \(Deb\)](#)

[Local Installer for Ubuntu22.04 cross-sbsa \(Deb\)](#)

CSDN @笑脸惹桃花

将得到的压缩文件进行解压，解压后得到下图三个文件夹，全选复制进 **cuda** 的文件夹中进行覆盖替换，替换完成后即 **cuda** 安装完成。按照本文教程安装

的 cuda 的文件夹默认在 C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v11.8 目录下。



3.2 创建虚拟环境

按下 Win 键，输入 anaconda prompt,打开下图所示的应用。



默认进入的是 base 环境，base 环境的版本与下载的 anaconda3 版本有关，因此不建议直接使用，需要新建环境，在新建环境之前建议更改默认的 pip 源和 conda 源可加速下载速度。

更改代码如下，直接输入即可，这里选择更换的是中科大源，建议使用。

1. `conda config --remove-key channels`
2. `conda config --add channels`
`https://mirrors.ustc.edu.cn/anaconda/pkgs/main/`
3. `conda config --add channels`
`https://mirrors.ustc.edu.cn/anaconda/pkgs/free/`

4. `conda config --add channels`
`https://mirrors.bfsu.edu.cn/anaconda/cloud/pytorch/`
5. `conda config --set show_channel_urls yes`
6. `pip config set global.index-url`
`https://mirrors.ustc.edu.cn/pypi/web/simple`

之后通过 `pip install` 包名方式安装的任何库(除了 pytorch 及相关库), 只要报错 **ERROR: No matching distribution found for 包名**, 在 `python` 版本与本文一致的情况下, 皆为源的访问速度慢导致的, 可以通过以下方法临时换源下载安装, 方法为 在安装包时通过 `-i` 参数指定镜像源, 可多试几个确定哪个速度最快最稳定

`pip install 包名 -i https://pypi.tuna.tsinghua.edu.cn/simple`
AI 写代码 bash

1. 清华大学
URL: `https://pypi.tuna.tsinghua.edu.cn/simple`
2. 阿里云
URL: `https://mirrors.aliyun.com/pypi/simple`
3. 中国科学技术大学 (USTC)
URL: `https://pypi.mirrors.ustc.edu.cn/simple`
4. 豆瓣 (Douban)
URL: `https://pypi.doubanio.com/simple`
5. 华为云
URL: `https://repo.huaweicloud.com/repository/pypi/simple`
6. 腾讯云
URL: `https://mirrors.cloud.tencent.com/pypi/simple`

如:

`pip install numpy==1.26.3 -i https://pypi.tuna.tsinghua.edu.cn/simple`

此时新建虚拟环境 (需要关闭加速软件), 这里创建一个名为 `yolov11`, `python` 版本为 `3.10` 的虚拟环境, 也可以修改为其他名, 本文所用为 `python3.10`, 同时本教程所用 `pytorch` 源及 `python` 版本只支持 `3.10`。

`conda create -n yolov11 python=3.10`

```
The following packages will be downloaded:

package-----|-----build-----|-----
bzip2-1.0.8             h2bbff1b_6           90 KB  https://mirrors.ustc.edu.cn/anaconda/pkg...
libffi-3.4.4            hd77b12b_1          122 KB  https://mirrors.ustc.edu.cn/anaconda/pkg...
pip-24.2                py310haa95532_0      2.5 MB  https://mirrors.ustc.edu.cn/anaconda/pkg...
python-3.10.15          h4607a30_1          16.2 MB https://mirrors.ustc.edu.cn/anaconda/pkg...
setuptools-75.1.0       py310haa95532_0      1.6 MB  https://mirrors.ustc.edu.cn/anaconda/pkg...
wheel-0.44.0            py310haa95532_0      138 KB  https://mirrors.ustc.edu.cn/anaconda/pkg...
xz-5.4.6                h8cc25b3_1           609 KB  https://mirrors.ustc.edu.cn/anaconda/pkg...
-----|-----
Total:                  21.3 MB

The following NEW packages will be INSTALLED:

bzip2                anaconda/pkg.../win-64::bzip2-1.0.8-h2bbff1b_6
ca-certificates      anaconda/pkg.../win-64::ca-certificates-2024.9.24-haa95532_0
libffi               anaconda/pkg.../win-64::libffi-3.4.4-hd77b12b_1
openssl              anaconda/pkg.../win-64::openssl-3.0.15-h827c3e9_0
pip                  anaconda/pkg.../win-64::pip-24.2-py310haa95532_0
python               anaconda/pkg.../win-64::python-3.10.15-h4607a30_1
setuptools            anaconda/pkg.../win-64::setuptools-75.1.0-py310haa95532_0
sqlite               anaconda/pkg.../win-64::sqlite-3.45.3-h2bbff1b_0
tk                   anaconda/pkg.../win-64::tk-8.6.14-h0416ee5_0
tzdata               anaconda/pkg.../noarch::tzdata-2024b-h04d1e81_0
vc                   anaconda/pkg.../win-64::vc-14.40-h2eaa2aa_1
vs2015_runtime       anaconda/pkg.../win-64::vs2015_runtime-14.40.33807-h98bb1dd_1
wheel                anaconda/pkg.../win-64::wheel-0.44.0-py310haa95532_0
xz                   anaconda/pkg.../win-64::xz-5.4.6-h8cc25b3_1
zlib                 anaconda/pkg.../win-64::zlib-1.2.13-h8cc25b3_1

Proceed ([y]/n)? _
```

CSDN @笑脸惹桃花

回车后出现新建环境提醒输入 y 继续，耐心等待全部下载完成后自动安装。

注意：此时如果报错

UnavailableInvalidChannel: HTTP 404 NOT FOUND for channel
anaconda/pkg.../free <<https://mirrors.ustc.edu.cn/anaconda/pkg.../free>>

The channel is not accessible or is invalid.

You will need to adjust your conda configuration to proceed.
Use `conda config --show channels` to view your configuration's current state,
and use `conda config --show-sources` to view config file locations.

等错误，需要修改隐藏文件 .condarc 文件的内容，具体路径为：c:\users\用户名\condarc

使用记事本打开，复制下面这一段进去替换掉原本所有的内容，替换完之后保存即可

1. channels:
2. - defaults
3. show_channel_urls: true
4. channel_alias: <http://mirrors.tuna.tsinghua.edu.cn/anaconda>
5. default_channels:
6. - <http://mirrors.tuna.tsinghua.edu.cn/anaconda/pkg.../main>
7. - <http://mirrors.tuna.tsinghua.edu.cn/anaconda/pkg.../free>
8. - <http://mirrors.tuna.tsinghua.edu.cn/anaconda/pkg.../r>
9. - <http://mirrors.tuna.tsinghua.edu.cn/anaconda/pkg.../pro>

```
10.     - http://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgsg/msys2
11.  custom_channels:
12.     conda-forge:
        http://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud
13.     msys2: http://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud
14.     bioconda:
        http://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud
15.     menpo: http://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud
16.     pytorch: http://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud
17.     simpleitk:
        http://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud
```

此时再次输入命令新建 **yolov11** 环境，本文所用为 **python3.10**，同时本教程所用 **pytorch** 源及 **python** 版本只支持 **3.10**。

```
conda create -n yolov11 python=3.10
AI 写代码 bash
```

创建完之后输入

```
conda activate yolov11
```

进入 **yolov11** 环境之中，此时有 **NVIDIA** 显卡安装 **GPU** 版看 **4.3**，其它需要安装 **CPU** 版看 **4.4**。

3.3 安装 GPU 版本

3.3.1 安装 pytorch（GPU 版）

打开 **prompt** 输入 **conda activate yolov11** 进入 **yolov11** 环境，之后输入下方命令即可安装 **pytorch**，耐心等待安装完成。

```
pip install torch==2.0.0+cu118 torchvision==0.15.1+cu118 --extra-
index-url https://download.pytorch.org/whl/cu118
AI 写代码 bash
```

出现 **ERROR: No matching distribution found for torch==2.0.0+cu118** 等是因为 **python** 版本不对！

torch 及相关库比较大，需要耐心等待下载完之后出现 **done** 则安装完成，因为是外网，如果下载不下来，可以点此下载 **torch** [夸克网盘下载](https://pan.quark.cn/s/fb79df20edb8) [\[https://pan.quark.cn/s/fb79df20edb8\]](https://pan.quark.cn/s/fb79df20edb8)，然后通过 **pip install** 安装本地文件的方式安装，可以参考下图的方式，将 **torch** 放入 **d** 盘的 **test** 文件夹下，通过 **cd** 将

文件目录跳转，再输入如下指令。（只支持 **python3.10** 版本，其余版本无法安装）

```
pip install torch-2.0.0+cu118-cp310-cp310-win_amd64.whl
```



```
(yolov11) C:\Users\ZYP>d:
(yolov11) D:\>cd test
(yolov11) D:\test>pip install torch-2.0.0+cu118-cp310-cp310-win_amd64.whl
```

耐心等待安装完成后，再输入下面的指令通过 **pip** 安装其余库。

```
pip install torch==2.0.0+cu118 torchvision==0.15.1+cu118 --extra-
index-url https://download.pytorch.org/whl/cu118
AI 写代码 bash
```

3.3.2 安装 ultralytics 库

这个库包含了几乎全部运行 **yolov11** 所需的环境，输入以下命令后耐心等待即可

```
pip install ultralytics
```

如果报错 **ERROR: No matching distribution found for ultralytics** 则输入以下指令安装

```
pip install ultralytics -i https://pypi.tuna.tsinghua.edu.cn/simple
```

也可以选择安装 **ultralytics** 的对应版本所需库，如 **pip install ultralytics==8.3.20**，上方安装不报错则默认不需要安装指定版本。安装完成出现下图则环境配置完成。

3.4 安装 CPU 版本

3.4.1 安装 pytorch（CPU 版）

有英伟达显卡的按照上述流程安装后即可跳过本部分，无英伟达显卡则需要按照下述方法进行，

打开 **prompt** ,进入 **yolov11** 环境之后，输入如下命令即可安装 **cpu** 版本的 **pytorch**

```
pip install torch==2.0.0 torchvision==0.15.1 torchaudio==2.0.1 --
index-url https://download.pytorch.org/whl/cpu
```

输入指令后看到下图再按下回车则开始正常安装。

```
libpng          anaconda/pkgs/free/win-64::libpng-1.6.17-0
libtiff         anaconda/pkgs/free/win-64::libtiff-4.0.2-1
libuv          anaconda/pkgs/main/win-64::libuv-1.48.0-h827c3e9_0
libwebp        anaconda/pkgs/main/win-64::libwebp-1.2.4-h2bbff1b_0
libwebp-base   anaconda/pkgs/main/win-64::libwebp-base-1.2.4-h2bbff1b_1
markupsafe     anaconda/pkgs/main/win-64::markupsafe-2.1.3-py310h2bbff1b_0
mkl            anaconda/pkgs/main/win-64::mkl-2023.1.0-h6b88ed4_46358
mkl-service    anaconda/pkgs/main/win-64::mkl-service-2.4.0-py310h2bbff1b_1
mkl_fft       anaconda/pkgs/main/win-64::mkl_fft-1.3.10-py310h827c3e9_0
mkl_random     anaconda/pkgs/main/win-64::mkl_random-1.2.7-py310hc64d2fc_0
mpc            anaconda/pkgs/main/win-64::mpc-1.1.0-h7edee0f_1
mpfr           anaconda/pkgs/main/win-64::mpfr-4.0.2-h62dcd97_1
mpir           anaconda/pkgs/main/win-64::mpir-3.0.0-hec2e145_1
mpmath         anaconda/pkgs/main/win-64::mpmath-1.3.0-py310haa95532_0
networkx       anaconda/pkgs/main/win-64::networkx-3.2.1-py310haa95532_0
numpy          anaconda/pkgs/main/win-64::numpy-2.0.1-py310h055cbcc_1
numpy-base    anaconda/pkgs/main/win-64::numpy-base-2.0.1-py310h65a83cf_1
pillow        anaconda/pkgs/main/win-64::pillow-9.3.0-py310hdc2b20a_1
pysocks       anaconda/pkgs/main/win-64::pysocks-1.7.1-py310haa95532_0
pytorch       pytorch/win-64::pytorch-2.5.0-py3.10_cpu_0
pytorch-mutex pytorch/noarch::pytorch-mutex-1.0-cpu
pyyaml        anaconda/pkgs/main/win-64::pyyaml-6.0.2-py310h827c3e9_0
requests     anaconda/pkgs/main/win-64::requests-2.32.3-py310haa95532_0
sympy        anaconda/pkgs/main/win-64::sympy-1.13.2-py310haa95532_0
tbb          anaconda/pkgs/main/win-64::tbb-2021.8.0-h59b6b97_0
torchaudio  pytorch/win-64::torchaudio-2.5.0-py310_cpu
torchvision pytorch/win-64::torchvision-0.20.0-py310_cpu
typing_extensions anaconda/pkgs/main/win-64::typing_extensions-4.11.0-py310haa95532_0
urllib3     anaconda/pkgs/main/win-64::urllib3-2.2.3-py310haa95532_0
win_inet_pton anaconda/pkgs/main/win-64::win_inet_pton-1.1.0-py310haa95532_0
yaml        anaconda/pkgs/main/win-64::yaml-0.2.5-he774522_0

Proceed ([y]/n)? _
```

CSDN @笑脸惹桃花

耐心等待安装完成之后安装 ultralytics 库。

3.4.2 安装 ultralytics 库

```
pip install ultralytics
```

如果报错 **ERROR: No matching distribution found for ultralytics** 则输入以下指令安装

```
pip install ultralytics -i https://pypi.tuna.tsinghua.edu.cn/simple
```

也可以选择安装 ultralytics 的对应版本所需库，如 `pip install ultralytics==8.3.20`，上方安装不报错则默认不需要安装指定版本。安装完成出现下图则环境配置完成。

```
Installing collected packages: pytz, py-cpuinfo, zipp, tzdata, sympy, six, scipy, pyparsing, psutil, packaging,
opencv-python, kiwisolver, fsspec, fonttools, cycler, contourpy, colorama, tqdm, python-dateutil, importlib-res
ources, ultralytics-thop, pandas, matplotlib, seaborn, ultralytics
Attempting uninstall: sympy
  Found existing installation: sympy 1.13.2
  Uninstalling sympy-1.13.2:
    Successfully uninstalled sympy-1.13.2
Successfully installed colorama-0.4.6 contourpy-1.3.0 cycler-0.12.1 fonttools-4.54.1 fsspec-2024.10.0 importlib-
resources-6.4.5 kiwisolver-1.4.7 matplotlib-3.9.2 opencv-python-4.10.0.84 packaging-24.1 pandas-2.2.3 psutil-6.1
.0 py-cpuinfo-9.0.0 pyparsing-3.2.0 python-dateutil-2.9.0.post0 pytz-2024.2 scipy-1.13.1 seaborn-0.13.2 six-1.16
.0 sympy-1.13.1 tqdm-4.66.5 tzdata-2024.2 ultralytics-8.3.23 ultralytics-thop-2.0.9 zipp-3.20.1
```

CSDN @笑脸惹桃花

4. 源码及使用环境

4.1 下载 YOLOv11 源码

YOLOv11 (ultralytics) 源码地址:

<https://github.com/ultralytics/ultralytics>/
<https://github.com/ultralytics/ultralytics/>

点进去下载代码，现在还在持续更新，这里使用 v8.3.20 版本演示，进不去可能需要科学上网，或者点击下载 [夸克网盘下载](#)

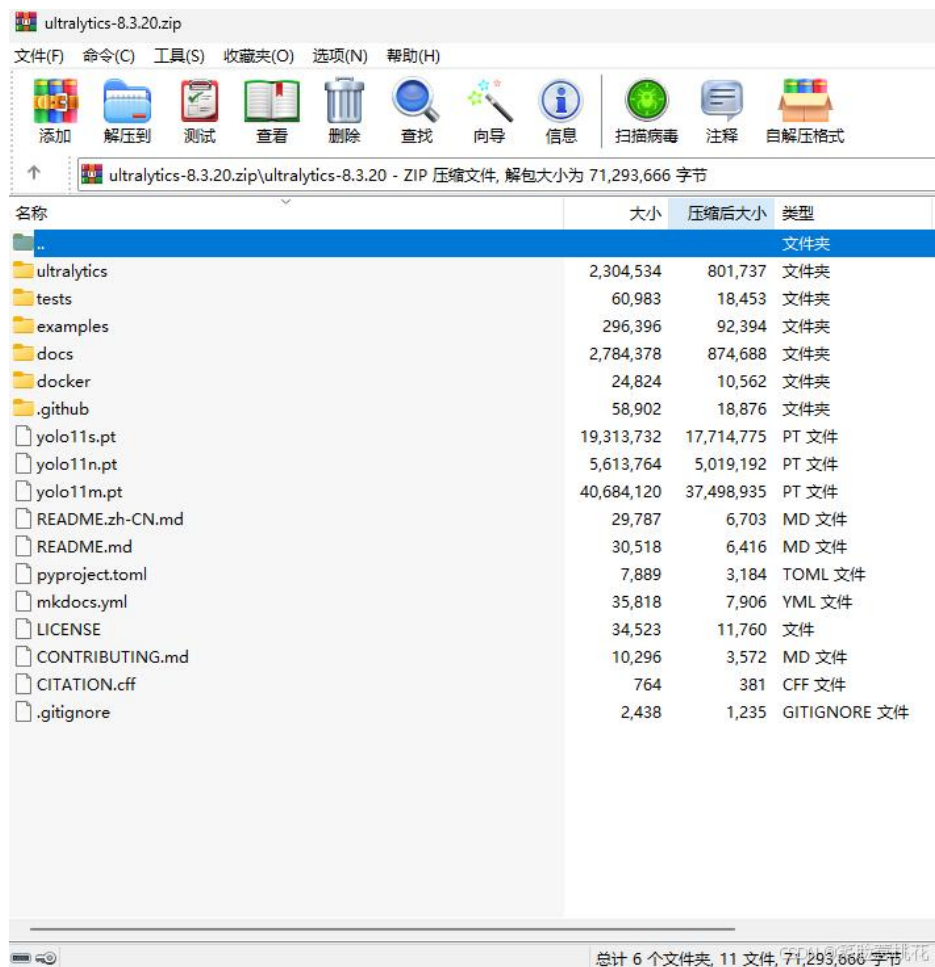
<https://pan.quark.cn/s/eb8cceb3df8a>，我上传了多版本可以全部转存避免后续找不到，cat 图片一并上传，下载 8.3.0 及以上版本，（压缩包内附带 yolov11n.pt、yolov11s.pt 和 yolov11m.pt 预训练权重，常规使用 yolov11n.pt 即可。

▼ Detection (COCO)

See [Detection Docs](#) for usage examples with these models trained on [COCO](#), which include

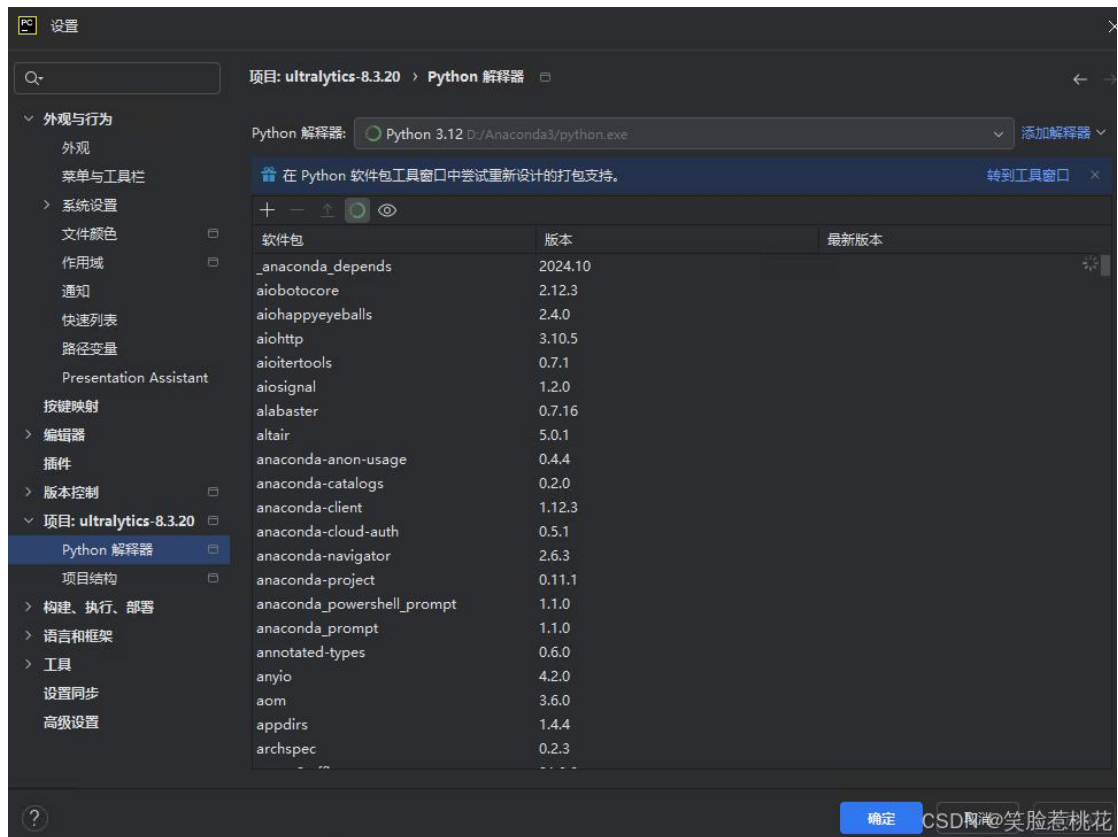
Model	size (pixels)	mAP ^{val} 50-95	Speed CPU ONNX (ms)	Speed T4 TensorRT10 (ms)	params (M)	FLOPs (B)
YOLO11n	640	39.5	56.1 ± 0.8	1.5 ± 0.0	2.6	6.5
YOLO11s	640	47.0	90.0 ± 1.2	2.5 ± 0.0	9.4	21.5
YOLO11m	640	51.5	183.2 ± 2.0	4.7 ± 0.1	20.1	68.0
YOLO11l	640	53.4	238.6 ± 1.4	6.2 ± 0.1	25.3	86.9
YOLO11x	640	54.7	462.8 ± 6.7	11.3 ± 0.2	56.9	194.9

CSDN @笑臉惹桃花

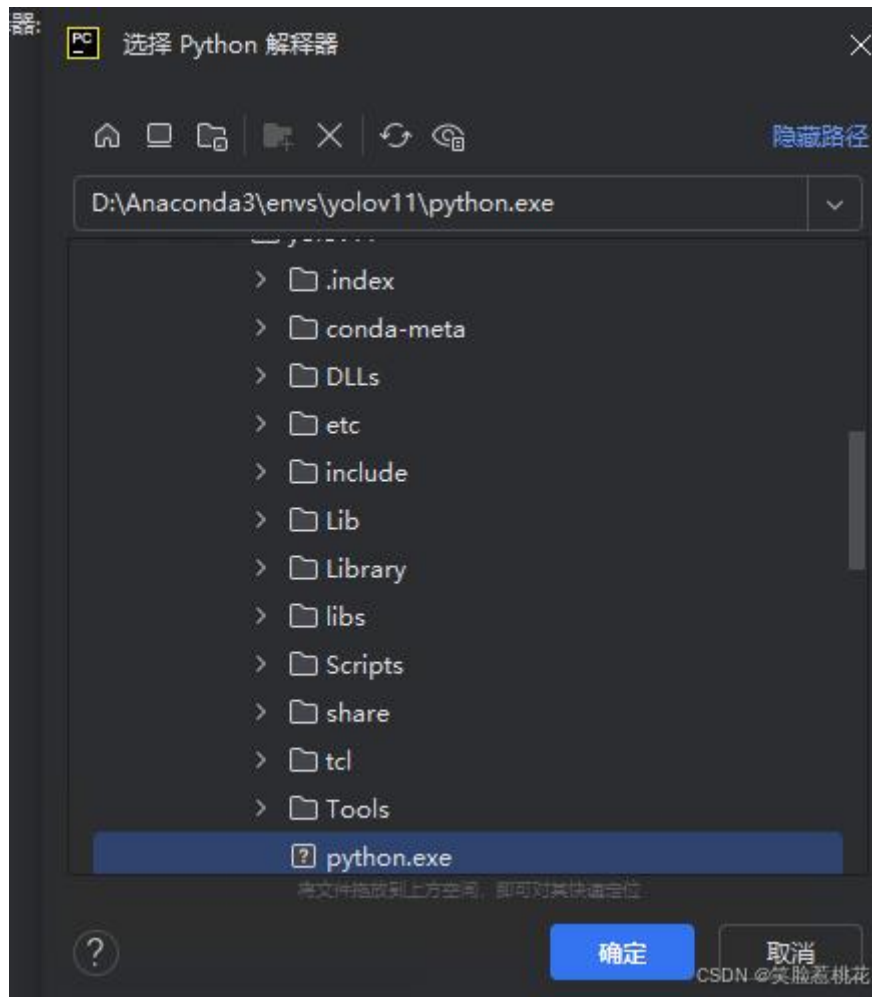


4.2 pycharm 导入环境

下载完成之后解压到 D 盘或其它盘文件夹内，此时点击鼠标右键文件夹通过 pycharm 打开，打开后需要配置虚拟环境，新版 pycharm 可选中文语言，点击文件-设置，点击 项目：ultralitics-8.3.20，点击 python 解释器，点击右边添加解释器-添加本地解释器，



点击 Virtualenv 环境 - 现有，点击右边三个点，找到刚才添加的 yolov11 环境的位置，按照本文配置即是 D:\Anaconda3\envs\yolov11\python.exe 路径，之后一直点确定，点应用，再点确定即可。



或者新版本的界面更为简单，按照下图方式选择就可以



选择 python，接着选择刚刚添加的 yolov11 环境，选择 python.exe，

如果找不到环境所在位置，可以输入

```
conda env list
```

运行

查看环境所在位置

```
(yolov11) C:\Users\030>conda env list

# conda environments:
#
base                  C:\Users\030\anaconda3
yolov11               * D:\anaconda3\envs\yolov11
```

CSDN @笑脸惹桃花

并寻找后选择 python.exe。

下图为环境正在加载，耐心等待完成即可，等待环境加载完成即可运行代码。



5. 验证环境

下载完成后将权重文件复制到 **ultralytics** 根目录下，即本文的 **ultralytics-8.3.20** 目录下，此时可以去网上下载一只猫猫的 **jpg** 图片（或者其它图片，根据自己喜好更改），修改文件名为 **cat.jpg**。检测环境是否有问题可以在 **prompt** 里 **yolov11** 环境下运行。注意预训练权重的文件名与代码中的文件名对应，看清楚文件名，**FileNotFoundError: xxxx does not exist** 类似这种报错都是文件没找到，自己细心一点就能避免。运行下面的指令需要 **cd** 到 **ultralytics-8.3.20** 目录，不然会报错。

```
yolo predict model=yololln.pt source='cat.jpg'
```

运行

输出下图即环境正常。

```
image 1/1 D:\ultralytics-8.3.20\cat.png: 416x640 1 cat, 78.5ms
Speed: 0.0ms preprocess, 78.5ms inference, 0.0ms postprocess per image at shape (1, 3, 416, 640)
Results saved to runs\detect\predict2
```

CSDN @笑脸惹桃花

或者通过前边安装的 **pycharm** 运行检测，新建一个 **yolov11_predict.py**，在文件中添加如下代码

1. `from ultralytics import YOLO`
2. `# 加载预训练的 YOLOv11n 模型`
3. `model = YOLO('yololln.pt')`
4. `source = 'cat.jpg' #更改为自己的图片路径`
5. `# 运行推理，并附加参数`
6. `model.predict(source, save=True)`

运行

运行之后如下即环境正常。

```
image 1/1 D:\ultralytics-8.3.20\cat.png: 416x640 1 cat, 88.8ms
Speed: 3.0ms preprocess, 88.8ms inference, 1.0ms postprocess per image at shape (1, 3, 416, 640)
Results saved to runs\detect\predict3

进程已结束，退出代码为 0
```

CSDN @笑脸惹桃花

如果报错 `ImportError: DLL load failed while importing _imaging`: 找不到指定的模块，是 pillow 版本问题导致的错误。

```
(yolov11) D:\ultralytics-8.3.20>yolo predict model=yolov8n.pt source='cat.jpg'
Traceback (most recent call last):
  File "D:\Anaconda3\envs\yolov11\lib\runpy.py", line 197, in _run_module_as_main
    return _run_code(code, main_globals, None,
  File "D:\Anaconda3\envs\yolov11\lib\runpy.py", line 87, in _run_code
    exec(code, run_globals)
  File "D:\Anaconda3\envs\yolov11\Scripts\yolo.exe\__main__.py", line 4, in <module>
  File "D:\Anaconda3\envs\yolov11\lib\site-packages\ultralytics\__init__.py", line 11, in <module>
    from ultralytics.models import NAS, RTDETR, SAM, YOLO, FastSAM, YOLOWorld
  File "D:\Anaconda3\envs\yolov11\lib\site-packages\ultralytics\models\__init__.py", line 3, in <module>
    from .fastsam import FastSAM
  File "D:\Anaconda3\envs\yolov11\lib\site-packages\ultralytics\models\fastsam\__init__.py", line 3, in <module>
    from .model import FastSAM
  File "D:\Anaconda3\envs\yolov11\lib\site-packages\ultralytics\models\fastsam\model.py", line 5, in <module>
    from ultralytics.engine.model import Model
  File "D:\Anaconda3\envs\yolov11\lib\site-packages\ultralytics\engine\model.py", line 9, in <module>
    from PIL import Image
  File "D:\Anaconda3\envs\yolov11\lib\site-packages\PIL\Image.py", line 100, in <module>
    from . import _imaging as core
ImportError: DLL load failed while importing _imaging: 找不到指定的模块。
```

CSDN @笑脸惹桃花

输入如下指令降低 pillow 的版本即可解决此问题。

```
pip install pillow==8.4.0
```

如果报错中有 numpy 相关的语句，如 `RuntimeError: Numpy is not available ; compiled using NumPy 1.x cannot be run in NumPy 2.1.2 as it may crash. To support both 1.x and 2.x versions of NumPy, modules must be compiled with NumPy 2.0.` 等等，可以将 numpy 降级，输入如下代码即可解决此类问题。

```
pip install numpy==1.26.3
```

运行

可以顺着路径查看检测后的图片，出现检测框则环境可以正常使用。

cat 0.80



CSDN @笑臉惹桃花

超详细目标检测：YOLOv11(ultralytics)训练自己的数据集、测试、评估、可视化界面

1. 训练模型

需要下载源码，这里选择的是 yolov11 v8.3.20 版本的，本文演示所用的安全帽检测数据集[点此下载 \[https://pan.quark.cn/s/2f5a0ce67843\]](https://pan.quark.cn/s/2f5a0ce67843)，注意，此数据集的两个标签分别为 'person','hat' 。

有了源码之后需要修改里面的参数，导入自己的数据集。

1.1 创建 data.yaml

在 yolov11 根目录下创建一个新的 data.yaml 文件，文件名可以变但是后缀需要为.yaml，内容如下，文件夹路径分别修改为前边划分后数据集的路径。test 可有可无

```
1. train: f:/VOCdevkit/train/images # train images (relative to
   'path') 128 images
2. val: f:/VOCdevkit/val/images # val images (relative to 'path')
   128 images
3. test: f:/VOCdevkit/test/images
4.
5. nc: 2
6.
7. # Classes
8. names: ['hat', 'nohat']
```

其他路径和类别自己替换，需要和上面数据集转换那里类别顺序一致。

1.2 训练模型

这是使用官方提供的预训练权重进行训练，使用 yolov11n.pt，也可以使用 yolov11s.pt，模型大小 $n < s < m < l < x$ ，训练时长成倍增加，官方的命名为 yolo11，本文 yolov11 与 yolo11 指代相同。

▼ Detection (COCO)

See [Detection Docs](#) for usage examples with these models trained on [COCO](#), which include

Model	size (pixels)	mAP ^{val} 50-95	Speed CPU ONNX (ms)	Speed T4 TensorRT10 (ms)	params (M)	FLOPs (B)
YOLO11n	640	39.5	56.1 ± 0.8	1.5 ± 0.0	2.6	6.5
YOLO11s	640	47.0	90.0 ± 1.2	2.5 ± 0.0	9.4	21.5
YOLO11m	640	51.5	183.2 ± 2.0	4.7 ± 0.1	20.1	68.0
YOLO11l	640	53.4	238.6 ± 1.4	6.2 ± 0.1	25.3	86.9
YOLO11x	640	54.7	462.8 ± 6.7	11.3 ± 0.2	56.9	194.9

CSDN 博客园 桃花

下载完成之后放入 **ultralytics-8.3.20** 根目录中，创建一个 **yolov11_train.py** 文件，内容如下，关闭 **amp** 训练。

```
1. import warnings
2. warnings.filterwarnings('ignore')
3. from ultralytics import YOLO
4. if __name__ == '__main__':
5.     model = YOLO('ultralytics/cfg/models/11/yolov11n.yaml')
6.     model.load('yolov11n.pt') #注释则不加载
7.     results = model.train(
8.         data='data.yaml', #数据集配置文件的路径
9.         epochs=200, #训练轮次总数
10.        batch=16, #批量大小，即单次输入多少图片训练
11.        imgsz=640, #训练图像尺寸
12.        workers=8, #加载数据的工作线程数
13.        device= 0, #指定训练的计算设备，无 nvidia 显卡则改为
        'cpu'
14.        optimizer='SGD', #训练使用优化器，可选
        auto,SGD,Adam,AdamW 等
15.        amp= True, #True 或者 False，解释为：自动混合精度(AMP)
        训练
16.        cache=False # True 在内存中缓存数据集图像，服务器推荐开
        启
17.    )
```

运行

这里用哪个模型对应哪个 **yaml**，如果使用 **yolov11s.pt** 则对应 **yolo11s.yaml**

epochs 是训练轮数，可以由少变多看训练效果，**workers** 和 **batch** 根据电脑性能进行调整，如果运行吃力则相应降低，最好为 2 的 n 次方。没有显卡则把 **device=0** 修改为 **device='cpu'**。

也可以使用命令行执行训练

```
yolo task=detect mode=train model=yolol1n.yaml pretrained=yolol1n.pt
data=data.yaml epochs=200 imgsz=640 device=0 optimizer='SGD'
workers=8 batch=64 amp=False cache=False
运行
```

训练过程如图，耐心等待训练完成即可，训练完成后会生成.pt 权重文件，可以用来验证训练效果。

```
10 epochs completed in 0.065 hours.
Optimizer stripped from /root/.ultralytics/runs/detect/train10/weights/last.pt, 12.2MB
Optimizer stripped from /root/.ultralytics/runs/detect/train10/weights/best.pt, 12.2MB
```

2. 模型测试

找到之前训练的结果保存路径，创建一个 `yolov11_predict.py` 文件，内容如下

```
1. from ultralytics import YOLO
2. # 加载训练好的模型，改为自己的路径
3. model = YOLO('runs/detect/train/weights/best.pt') #修改为训练
   好的路径
4. source = 'test1.jpg' #修改为自己的图片路径及文件名
5. # 运行推理，并附加参数
6. model.predict(source, save=True)
```

运行

运行后就会得到预测模型结果

```
Results saved to D:/ultralytics/runs/detect/predict
Inference times for images are shown in the table below
D:\ultralytics\runs\detect\predict> python yolov11_predict.py
```

或者使用命令行指令进行预测，权重和图片路径自己修改。

```
yolo predict model=runs/detect/train/weights/best.pt
source='test1.jpg'
```

测试集上推理模型精度代码如下，可新增 `v11_val.py`，输入下方代码，更改模型路径及数据集路径即可。

```
1. import warnings
2. warnings.filterwarnings('ignore')
3. from ultralytics import YOLO
4. if __name__ == '__main__':
```

```

5.     model = YOLO('runs/detect/train/weights/best.pt') #修改为
      自己训练的模型路径
6.     model.val(data='hat.yaml', #修改为自己的数据集 yaml 文件
7.              split='test',
8.              imgsz=640,
9.              batch=16,
10.             iou=0.6, #阈值可以改, mAP50 为 0.5 的情况下
11.             conf=0.001,
12.             workers=8,
13.             )

```

运行



运行后即出现测试集上各类别的精度及总体精度。

3. 可视化界面

很多同学的需求是制作出一个可视化界面 ui 作为系统来展示预测的效果, 这里我分享了两个简单的图片预测的界面, 导入模型权重文件和图片就可以进行预测并展示, pyqt5 写的可以参考这篇文章 [点击这里](https://blog.csdn.net/qq_67105081/article/details/147745193?spm=1001.2014.3001.5501)

[\[https://blog.csdn.net/qq_67105081/article/details/147745193?spm=1001.2014.3001.5501\]](https://blog.csdn.net/qq_67105081/article/details/147745193?spm=1001.2014.3001.5501), 界面如下

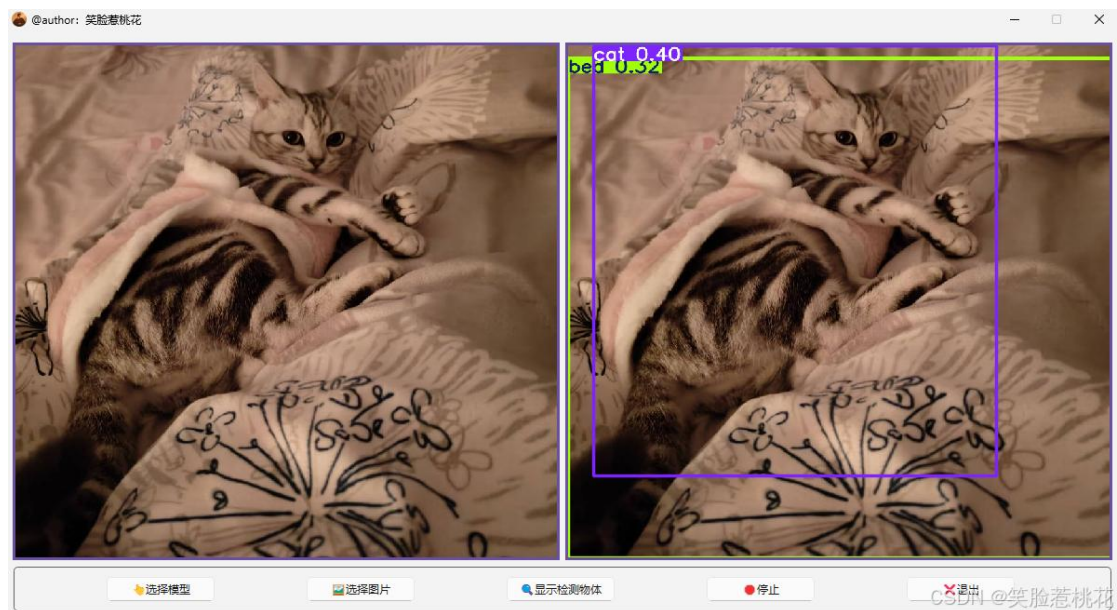




pyside6 可以参考 [这篇文章](https://blog.csdn.net/qq_67105081/article/details/144974796?spm=1001.2014.3001.5502)

[\[https://blog.csdn.net/qq_67105081/article/details/144974796?spm=1001.2014.3001.5502\]](https://blog.csdn.net/qq_67105081/article/details/144974796?spm=1001.2014.3001.5502) 或 [这篇文章](https://blog.csdn.net/qq_67105081/article/details/146458458?spm=1001.2014.3001.5501)

[\[https://blog.csdn.net/qq_67105081/article/details/146458458?spm=1001.2014.3001.5501\]](https://blog.csdn.net/qq_67105081/article/details/146458458?spm=1001.2014.3001.5501), 效果如下:





免费的功能较为简单，只有图片检测显示。

